

The SNN tool

ar011 · 6 July 2026

One command-line tool, *tools/snn/tool.py*, drives every simulation, training run, and measurement in this project. Experiment runners never import the model code; they shell out to this tool, which writes data files (metrics, rasters, population traces, weight dumps) into an output directory, and then draw their own figures from those files. The tool is the engine; the plotting lives in the runners under *experiments/*.

This page is the reference for that engine. It is not a tutorial. It is the dictionary you reach for when an experiment mentions *-ei-strength 0.5* and you want to know exactly what that did.

At a glance

- Run it with *uv run python tools/snn/tool.py* (never a bare *python*; the repo convention is *uv*).
- Three subcommands: *sim* (forward pass plus metrics), *train* (surrogate-gradient BPTT), *dump-weights* (emit weight matrices).
- Every parameter is a flag. There is no global config file; a saved run's *config.json* can be inherited with *-load-config*.
- Every run writes *config.json*, *run.sh*, *output.log*, and *run.jsonl* for reproducibility.
- The tool emits data, not figures. The runner draws the figure.

The tool and the experiment

demolab draws a hard line between a **tool** and an **experiment**. A tool holds the reusable science and speaks only through files; an experiment (a runner in *experiments/expNNN.py*) chooses which tool commands to run, then reads their data files back and renders the figures. The runner reaches the tool by running its CLI as a subprocess, never by importing it. That firewall is what keeps *tools/snn/* generic and lets the same engine serve every writeup in the lab.

So a typical experiment does three things: it invokes *tool.py* (often many times, sweeping a parameter), it aggregates each run's config plus headline metrics into a single *numbers.json*, and it draws PNG or SVG figures from the run's data. The committed record lands in *artifacts/data/expNNN/*; the tool's own scratch output is disposable (see Artifacts).

Quick start

Run a metrics-only forward pass of the canonical PING network:

```
uv run python tools/snn/tool.py sim --model ping --ei-strength 0.5
```

Train on MNIST for 100 epochs:

```
uv run python tools/snn/tool.py train --dataset mnist --epochs 100 --lr 0.0001
```

Evaluate a trained run on the test set, replaying it at a coarser timestep:

```
uv run python tools/snn/tool.py sim --infer \
  --load-config runs/foo/config.json \
  --load-weights runs/foo/weights.pth \
  --dt 0.5
```

Each subcommand has its own complete *-help*, and that per-subcommand help is authoritative. The top-level dispatcher’s group summary is hand-maintained and can lag the parser. When a flag here disagrees with reality, run *sim -help* and trust that.

Commands

sim

Run one forward pass and report firing-rate metrics. This is the cheapest mode (no training, no plots), used by the test suite, the dt-stability checks in ar003, and every experiment that needs a single inference pass over a trained or fresh network.

```
uv run python tools/snn/tool.py sim --model ping --dataset mnist --digit 3
```

On its own it prints metrics. The flags below make it load trained weights, evaluate a test set, emit extra data artifacts, or inject perturbations. There is no *-image* or *-video*: where those retired flags once produced panels and sweep MP4s, *sim* now emits raw data (via *-outputs*) that the calling runner plots, and sweep videos are assembled runner-side from many *sim* calls.

flag	default	description
<i>-infer</i>	off	Load trained weights and evaluate test-set accuracy; writes <i>results.json</i> (and <i>metrics.json</i>).
<i>-load-config PATH</i>	—	Load a saved <i>config.json</i> and inherit its model, dataset, and parameters. Explicit CLI flags override loaded values.
<i>-load-weights PATH</i>	—	Path to a <i>weights.pth</i> file for inference.

<code>-max-samples INT</code>	all	[<i>-infer</i>] Cap the evaluation set to N samples.
<code>-outputs OUTPUT [dots.c]</code>	—	[<i>-infer</i>] Extra artifacts from the single forward pass (<i>metrics.json</i> always written): <i>per_cell_rates</i> (per-cell E/I Hz to <i>per_cell_rates.npz</i>), <i>pop_traces</i> (per-trial population activity to <i>pop_traces.npz</i> , base signal for PSD / f_γ), <i>rasters</i> (sparse spike indices to <i>rasters.npz</i> , for cycle-level analysis).
<code>-tau-gaba FLOAT</code>	inherited / 9.0	[<i>-infer</i>] Override τ_{GABA} (ms) to replay a trained cell under specified inhibitory dynamics. Normally unset: <i>-load-config</i> inherits the trained value.
<code>-skip-load PREFIX [dots.c]</code>	—	[<i>-infer</i>] Drop <i>state_dict</i> keys with these prefixes before loading (e.g. <i>W_ei</i> . <i>W_ie</i> .) so a fresh sub-block survives. Transfer-load probes (exp038).
<code>-perturb-mode {drop, add}</code>	—	[<i>-infer</i>] Hidden-spike perturbation inside the forward loop: <i>drop</i> (Bernoulli mask), <i>add</i> (Poisson Hz). The exp037 drop/add asymmetry.
<code>-perturb-level LEVEL [dots.c]</code>	—	[<i>-perturb-mode</i>] One value: probability for <i>drop</i> , Hz for <i>add</i> .

<i>-i-override-file PATH</i>	—	[<i>-infer</i>] NPZ with a sparse per-trial I-spike stream to substitute for the inhibitory spikes each timestep. Injection dual of <i>-outputs rasters</i> (exp042).
<i>-input-file PATH</i>	—	NPZ with <i>input_spikes</i> (T, B, N_IN) to forward instead of Poisson input. Arbitrary stimulus (exp048 digit streams).
<i>-scale-w-in / -scale-w-ei / -scale-w-ie FLOAT</i>	1.0	[<i>-infer</i>] Multiply loaded input / E→I / I→E weights before the forward pass. Inference-time coupling sweeps without retraining (exp038).
<i>-sample-index INT</i>	—	Raw test-set index for a snapshot, overriding <i>-digit / -sample</i> .
<i>-n-in / -n-inh / -n-batch INT</i>	784 / — / 64	[synthetic-spikes] Input channels, inhibitory pool size, Poisson trials averaged.
<i>-w-ei-mean / -w-ie-mean FLOAT</i>	from <i>-ei-strength</i>	[synthetic-spikes] Explicit W_EI / W_IE mean (std = $0.1 \cdot \text{mean}$).
<i>-private-w-in</i>	off	[synthetic-spikes] Identity W_in: one input channel per E cell.

The block from *-skip-load* down is the **generic-primitive family**: small, experiment-agnostic hooks (perturb hidden spikes, inject an inhibitory stream, forward an arbitrary input file, scale a weight block at inference). Runners compose these instead of importing model code, and that is what keeps the tool/experiment boundary clean.

train

Surrogate-gradient BPTT training loop. Writes *weights.pth*, *metrics.json*, a per-step *metrics.jsonl*, and *test_predictions.json*.

```
uv run python tools/snn/tool.py train --model ping --dataset mnist \
  --epochs 100 --lr 0.0001 --v-grad-dampen 1000
```

`--epochs 0` runs the init snapshot only, useful as a probe.

flag	default	description
<code>--lr FLOAT</code>	0.01	Adam learning rate. Biophysical models (ping) typically need 0.0001; current-based models 0.01.
<code>--epochs INT</code>	0	Number of epochs. 0 = init-snapshot probe only.
<code>--batch-size INT</code>	64	DataLoader batch size.
<code>--max-samples INT</code>	all	Cap dataset to N samples for smoke tests.
<code>--v-grad-dampen FLOAT</code>	80.0	Dampening factor d on the COBA membrane-voltage gradient. PING needs a much larger value ($d = 1000$ in the paper) to stabilise BPTT through the E $\leftrightarrow$ I loop; COBA trains at the default. Theory in ar006.
<code>--fr-reg-upper-target-hz FLOAT</code>	0	Population-mean hidden-E firing-rate ceiling per presentation, in Hz. The one-sided squared overshoot is averaged over presentations and hidden layers; active only when the strength is positive.
<code>--fr-reg-upper-strength FLOAT</code>	0	Coefficient s_u on the upper regulariser. Cramer et al.: 0.06.
<code>--tau-gaba FLOAT</code>	9.0 ms	Override τ_{GABA} . Default = <code>models.py</code> 's value (Börger / Buzsáki-Wang range). exp041 sweeps this across {4.5 ... 27} ms while training PING from scratch; the realised gamma frequency f_γ tracks $1/\tau_{\text{GABA}}$.

dump-weights

Build the network from a config and emit its weight matrices to `weights_dump.npz`: the init state, plus (with `--load-weights`) the trained state. It runs no forward pass.

```
uv run python tools/snn/tool.py dump-weights \
  --load-config runs/foo/config.json \
  --load-weights runs/foo/weights.pth \
  --out-dir runs/foo/dump
```

Keys follow `W_ff_N_init` / `W_ff_N_trained` (feedforward, per layer N) plus the E-I blocks `W_ei` / `W_ie`. This is how a runner recovers the trained readout matrix (`W_out` = the last `W_ff`) or compares init-vs-trained loop weights (the exp049 pruning analysis) without loading the model in-process. It takes the shared option groups plus `--load-config` / `--load-weights`.

Shared options

These groups are attached to every subcommand. The grouping matches what each subcommand's *-help* prints.

Network

flag	default	description
<i>-model {ping}</i>	<i>ping</i>	Architecture. <i>ping</i> is the COBANet with E $\leftrightarrow$ I coupling; with <i>-ei-strength 0</i> the inhibitory loop is silenced for a no-rhythm control.
<i>-n-hidden INT [INT dots.c]</i>	dataset-dependent	Hidden layer sizes. One integer = single layer; multiple stacks layers. Default for mnist: 1024.
<i>-readout {rate, mem-mean}</i>	<i>rate</i>	Output stage. <i>rate</i> sums last-hidden spikes and projects linearly at the final timestep. <i>mem-mean</i> averages the pre-reset voltage of a spiking, subtractively resetting output LIF over time (the trained classification readout).
<i>-dales-law / -no-dales-law</i>	on	Enforce Dale's law (non-negative weights) or allow signed weights. <i>-no-dales-law</i> is used for balanced-network experiments.
<i>-ei-strength FLOAT</i>	0.5	E-I coupling strength <i>s</i> . Sets $W_{EI} = s$ and $W_{IE} = s \cdot \text{ratio}$.
<i>-ei-ratio FLOAT</i>	2.0	W_{IE} / W_{EI} .
<i>-w-in-initial-zero-fraction FLOAT</i>	0.95	Fraction of input parameters set to zero at initialization. All remain trainable and may regrow.
<i>-recurrent-initial-zero-fraction FLOAT</i>	0.0	Fraction of recurrent parameters set to zero at initialization; survivors are rescaled by $1/(1-s)$. Trainable matrices may become dense.
<i>-exact-k-initialization</i>	off	Choose exactly $K = \text{round}((1-s) \cdot N_{\text{pre}})$ initially

		non-zero recurrent entries per post cell instead of Bernoulli zeroing. This does not impose a persistent connectivity mask.
<i>-dt FLOAT</i>	0.25	Integration timestep (ms).
<i>-t-ms FLOAT</i>	200	Total trial duration (ms). Metrics are measured over the full trace; runners strip any startup transient in post.
<i>-readout-w-out-scale FLOAT</i>	1.0	Scalar applied to the readout matrix after <i>build_net</i> , compensating for low hidden firing rate under <i>mem-mean</i> . Train-mode only.
<i>-surrogate-slope FLOAT</i>	1.0	Fast-sigmoid surrogate-gradient slope β . Larger = narrower active window. Cramer et al. use 40 for SHD RSNNs.

The drive family below also lives in the Network group. It exists for the balanced-network (Brunel / van Vreeswijk-Sompolinsky) experiments and the Lyapunov chaos probe; canonical PING runs leave all of it off.

flag	default	description
<i>-independent-drive RATE G_PER_SPIKE</i>	off	Per-E-cell independent Poisson drive (bypasses W_in): N_E uncorrelated streams at RATE Hz, each spike adding G_PER_SPIKE μ S of g_E. Zero cross-cell correlation.
<i>-independent-drive-i RATE G_PER_SPIKE</i>	off	As above, targeting the I population directly. Needed for the full V&S asynchronous-irregular state.
<i>-quenched-drive MEAN STD</i>	off	Per-E-cell DC conductance drawn once from N(MEAN, STD) μ S and frozen for the trial. V&S quenched input: no fluctuation, so it cannot pin spike times; the Lyapunov probe then measures autonomous chaos.

<i>-quenched-drive-i MEAN STD</i>	off	Per-I-cell frozen DC conductance.
<i>-lyapunov-eps FLOAT</i>	0 (off)	If > 0 (synthetic-spikes mode), rerun with all membranes ε -perturbed at $t=0$ and save the divergence $\ \Delta V(t)\ $ to <i>snapshot.npz</i> . Its growth rate is the max Lyapunov exponent: positive for the chaotic V&S state, ≈ 0 for cycle-locked PING.

Input

flag	default	description
<i>-input {synthetic-spikes, dataset}</i>	<i>synthetic-spikes</i>	Stimulus regime. <i>synthetic-spikes</i> is Poisson at <i>-input-rate</i> . <i>dataset</i> draws from <i>-dataset</i> .
<i>-input-rate FLOAT</i>	25	Baseline input rate (Hz).
<i>-digit INT</i>	0	Dataset class (0–9).
<i>-sample INT</i>	0	Sample index within the class.
<i>-sample-index INT</i>	—	Raw test-set index, overriding <i>-digit</i> / <i>-sample</i> .
<i>-dataset {mnist}</i>	<i>mnist</i>	Dataset. <i>mnist</i> is the full 28×28 image encoded to spikes.

Weights (advanced)

flag	default	description
<i>-w-in MEAN [STD]</i>	<i>0.3 0.06</i>	Input fan-in init. Single value sets $STD = MEAN \times 0.1$.
<i>-w-ei MEAN STD</i>	from <i>-ei-strength</i>	Override the W_{EI} init.
<i>-w-ie MEAN STD</i>	from <i>-ei-strength</i> / <i>-ei-ratio</i>	Override the W_{IE} init.
<i>-w-ii MEAN STD</i>	<i>0 0</i>	W_{II} (I→I) init. Off by default (canonical PING has no I→I). Enable for balanced-network experiments.
<i>-w-ee MEAN STD</i>	<i>0 0</i>	W_{EE} (E→E) init. Off by default. Enable for the full four-coupling balanced network, where recurrent excitation pins the E rate.
<i>-trainable-w-ei</i>	frozen	Promote E→I to gradient-carrying. Asks whether the optimiser will discover the PING-loop weights from scratch.
<i>-trainable-w-ie</i>	frozen	Promote I→E. The exp049 result <i>gradient descent dismantles PING</i>

		comes from flipping <i>-trainable-w-ei</i> and <i>-trainable-w-ie</i> on simultaneously.
--	--	--

Output

flag	default	description
<i>-out-dir DIR</i>	<i>temp/pinglab-cli/</i>	Output directory. The default is scratch (gitignored); runners always pass an explicit path.
<i>-wipe-dir</i>	off	Clear the output directory before the run.

Execution

flag	default	description
<i>-seed INT</i>	—	RNG seed. Seeds Python, NumPy, torch (CPU + CUDA + MPS) before dataset load and model init. Persisted to <i>config.json</i> .
<i>-modal</i>	off	Re-dispatch to Modal.com. Artifacts sync back to <i>-out-dir</i> after completion.
<i>-modal-gpu {none, T4, L4, A10G, A100, H100}</i>	<i>T4</i>	GPU type for Modal runs. <i>none</i> runs CPU-only.

-modal costs money. The project default is local; only pass it when explicitly instructed.

Config inheritance

The trick that makes the experiment chain work is *-load-config*. Every *train* run writes a *config.json* alongside its *weights.pth*; a later *sim* or *dump-weights* run inherits from it:

```
uv run python tools/snn/tool.py sim --infer \
  --load-config runs/foo/config.json \
  --load-weights runs/foo/weights.pth \
  --dt 0.5
```

This inherits the model, hidden sizes, dataset, E-I parameters, input rate, τ_{GABA} , and seed, while the explicitly-passed *-dt 0.5* overrides the trained value, replaying the network at a new timestep. Precedence is: explicit CLI flag, then loaded config, then default. The parser builds the set of CLI-explicit flags from *sys.argv* before applying inheritance.

Backwards compatibility: old configs that stored *n_hidden* as a scalar are remapped to the *hidden_sizes* list, legacy model names are aliased with a one-line stderr note, and configs missing *dales_low* trigger a warning to pass it explicitly or retrain.

Artifacts

Every subcommand calls *save_run_artifacts* on entry, writing four provenance files into *-out-dir*:

file	contents
<i>config.json</i>	The parsed argparse namespace plus a provenance block (<i>git_sha</i> with a <i>dirty</i> suffix, <i>torch_version</i> , <i>device</i> , <i>python_env_hash</i> , <i>run_id</i> , <i>started_at</i>) and the <i>mode</i> . Consumed by <i>--load-config</i> and the runner's metadata extractors.
<i>run.sh</i>	The literal <i>sys.argv</i> joined with spaces and prefixed with a shebang. Re-running reproduces the run (modulo seeded RNG state, also in <i>config.json</i>).
<i>output.log</i>	The human run log. ANSI escapes are stripped from the file but preserved on stdout, so terminals see colour while the log stays grep-friendly.
<i>run.jsonl</i>	The machine-readable event spine: one typed JSON object per event (epoch rows, warnings, summary). This is what a runner parses when it wants structured progress, not scraped log text.

Each command adds its own outputs: *train* writes *weights.pth*, *metrics.json*, *metrics.jsonl*, *test_predictions.json*; *sim --infer* writes *metrics.json* (and *results.json*) plus whatever *--outputs* requested; *dump-weights* writes *weights_dump.npz*.

These files are scratch. By default they land under *temp/pinglab-cli/*, which is gitignored and overwritten every run. The committed record is produced by the runner: it aggregates each command's config plus headline metrics into one *artifacts/data/expNNN/numbers.json* and renders its figures alongside. That folder, not the ephemeral tool output, is what the publisher reads and what reaches the site.

Recipes

Train, then measure the trained network. *Train* writes a run directory; *sim --infer* reads it back and emits the population traces a runner needs for a PSD:

```
uv run python tools/snn/tool.py train --dataset mnist --epochs 100 \
    --lr 0.0001 --v-grad-dampen 1000 --out-dir runs/ping

uv run python tools/snn/tool.py sim --infer \
    --load-config runs/ping/config.json \
    --load-weights runs/ping/weights.pth \
    --outputs pop_traces per_cell_rates
```

Loop-transfer at inference (exp038). Take a network trained as COBA and scale its E→I coupling up at inference, with no retraining:

```
uv run python tools/snn/tool.py sim --infer \
    --load-config runs/coba/config.json \
    --load-weights runs/coba/weights.pth \
    --scale-w-ei 1.0 --scale-w-ie 1.0 --outputs rasters
```

Perturbation sweep (exp037). Drop a fraction of emitted spikes, or add off-phase Poisson noise, inside the forward loop:

```
uv run python tools/snn/tool.py sim --infer \  
  --load-config runs/ping/config.json --load-weights runs/ping/weights.pth \  
  --perturb-mode drop --perturb-level 0.8
```

Recover the trained readout matrix. Dump weights and read $W_{ff}N_{trained}$ (the last layer is W_{out}):

```
uv run python tools/snn/tool.py dump-weights \  
  --load-config runs/ping/config.json \  
  --load-weights runs/ping/weights.pth --out-dir runs/ping/dump
```